

一 库的作用和静态库的生成方法

库的作用

- 把常用函数打包复用：如数学、I/O、内存管理、字符串处理等，避免每个工程都重复实现。
- 改进两种原始做法的不足：
 - 方案1：把所有函数写进一个源文件并一起编译 → 可执行体臃肿、编译/链接慢、维护困难。
 - 方案2：每个函数单独成文件，让程序员显式挑选需要的目标文件链接 → 精细但麻烦，使用成本高。
- 静态库的思路：把一组相关的可重定位目标文件（.o）*合成一个带索引的*归档文件（.a）；链接器在解析外部符号时，会在这些归档里按需提取能“解引用”的成员目标文件，从而同时兼顾复用效率与使用便利。

静态库的生成方法

总流程：源文件 → 目标文件（.o） → 归档（.a），并为归档写入符号索引，便于链接器快速定位需要的成员。

把源文件编译为目标文件（不链接）

```
gcc -O2 -c atoi.c printf.c random.c # 生成 atoi.o printf.o random.o
```

用归档器 `ar` 把多个 .o 打包成 .a，并写入索引

```
ar rcs libmylib.a atoi.o printf.o random.o
# 说明：
# r 将成员写入/替换到归档
# c 若归档不存在则创建
# s 写入符号索引（旧系统可能需要：ranlib libmylib.a）
```

增量更新（库里某个函数改了，只需重编并替换对应 .o）

```
gcc -O2 -c printf.c # 只重编被修改的源文件 → printf.o
ar rcs libmylib.a printf.o # 用新的对象替换归档中的旧成员
```

（命名与配套）

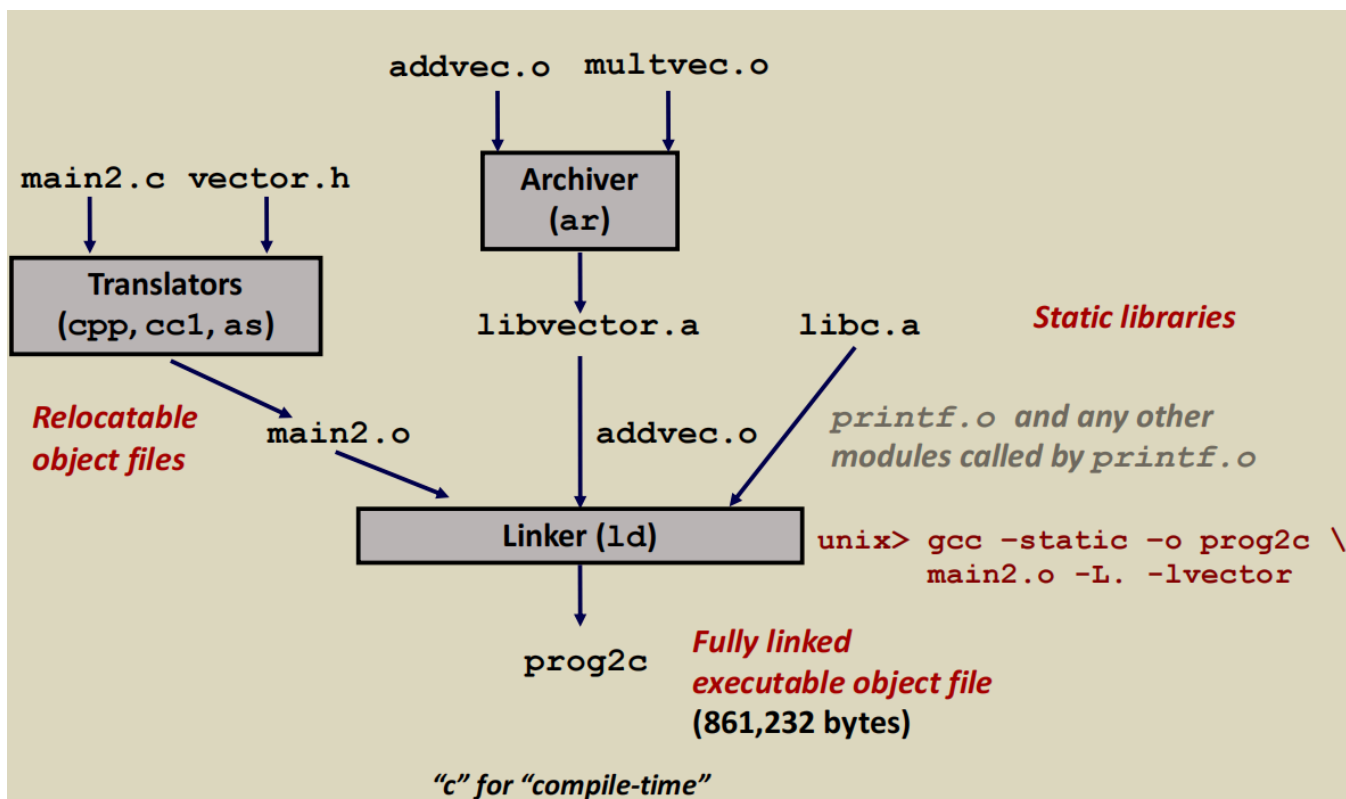
- 归档文件惯例命名为 `lib<name>.a`，搭配头文件（如 `mylib.h`）一同发布，供调用方 `#include`。
- 链接使用通常写作 `-L<库目录> -l<name>`（例如 `-L. -lmylib`）；（这属于 42-44 页的内容，这里仅作提示）。

二 解开若干常见库，展示其内容。

```
ar t C:/mingw64/lib/libbfd.a | sort -Object
archive.o
archive64.o
archures.o
bfd.o
bfdio.o
bfdwin.o
binary.o
cache.o
```

coff-bfd.o
coffgen.o
cofflink.o
compress.o
corefile.o
cpu-i386.o
cpu-iamcu.o
cpu-k10m.o
cpu-l10m.o
dwarf1.o
dwarf2.o
elf.o
elf32.o
elf32-gen.o
elf32-i386.o
elf64.o
elf64-gen.o
elf64-x86-64.o
elf-attrs.o
elf-eh-frame.o
elf-ifunc.o
elflink.o
elf-properties.o
elf-strtab.o
elf-vxworks.o
elfxx-x86.o
format.o
hash.o
ihex.o
init.o
libbfd.o
linker.o
merge.o
opncls.o
pe-i386.o
peigen.o
pei-i386.o
pei-x86_64.o
pex64igen.o
pe-x86_64.o
plugin.o
reloc.o
section.o
simple.o
srec.o
stabs.o
stab-syms.o
syms.o
targets.o
tekhex.o
verilog.o

三 静态库的基本原理和进行链接的主要过程



静态库

- **本质：**把一批可重定位目标文件（.o）用归档器 `ar` 打成一个归档文件（.a），并写入符号索引，方便链接器按需检索。
- **特点：**链接时，链接器只会从库里取用需要的成员 .o 合并进可执行文件；最终可执行体里包含这些代码的拷贝，运行时不再依赖库文件。

生成静态库

1. 把源文件编译成 .o（可重定位）

例： `addvec.c → addvec.o`， `multivec.c → multivec.o`；同时主程序 `main2.c → main2.o`（依赖 `vector.h`）。

2. 用 `ar` 归档为 .a

例： `ar rcs libvector.a addvec.o multivec.o`

这一步得到图中的 `libvector.a`（自定义向量库）。系统还提供 `libc.a`（C 标准库）。

3. 链接过程（图中部的大框 Linker）

```
gcc -static -o prog2c main2.o -L. -lvector
```

`-static`：强制使用静态库； `-L.`：在当前目录找库； `-lvector`：找 `libvector.a`

(a) 收集未解析符号（从左往右扫描）

- 先读 `main2.o`，发现有对 `addvec`、`printf` 等未定义引用（undefined symbols）。

(b) 按命令行顺序扫描库，按需取用成员

- 扫到 `-lvector` (即 `libvector.a`):
查索引发现 `addvec` 由 `addvec.o` 定义，于是只提取 `addvec.o` 合并到链接单元，`addvec` 被解析。
- 接着 (由 `gcc` 驱动) 还会把 `libc.a` 放到命令行尾部参与扫描：
为了解析 `printf`，从 `libc.a` 中只取出 `printf.o` 以及它直接需要的其它成员模块，一并合并。
- 若仍有新的未解析引用，就继续在后续的库里找；直到未解析列表清空或报“未定义引用”错误。

(c) 生成最终可执行文件

- 当所有外部符号都已解析，`ld` 对各段分配最终地址并打补丁 (重定位)，产出完全链接的可执行文件 `prog2c`

四 共享库（动态库）的基本原理和进行链接的主要过程

动态库原理

把常用代码做成可被多个进程共享映射的 `.so`，在装载时或运行时由动态链接器装入并完成符号解析与重定位。

装载时动态链接 (load-time)

- 可执行文件里有 `.interp` 指明动态链接器 (如 `ld-linux.so`)，`.dynamic` 里列出依赖库 (`DT_NEEDED`，如 `libc.so`)。
- 进程启动时，动态链接器把所需 `.so` 映射到进程地址空间，完成重定位，并设置 **PLT/GOT** 以支持延迟绑定 (**lazy binding**)：第一次调用目标函数时再解析真实地址、打补丁，之后直接跳转。

运行时动态链接 (run-time)

- 程序主动用 `dlopen()/dlsym()/dlclose()` 按需加载/查找/卸载 `.so`，适合插件、A/B、按需加载等场景。

共享

- 同一 `.so` 的代码段可被多个进程共享映射，显著节省内存与缓存开销。

链接的主要过程

1. 把源文件编译为位置无关代码 (PIC)

```
gcc -Og -c addvec.c multvec.c -fpic # 或 -fPIC
```

1. 把若干 .o 链成共享库

```
gcc -shared -o libvector.so addvec.o multvec.o
```

产物是 `libvector.so` (动态向量库)。

应用如何与共享库链接

- 装载时依赖

```
# 链接时只记录“依赖关系”，不拷贝库代码到可执行体
gcc -o prog21 main2.o -L. -lvector      # 或直接写 ./libvector.so
```

生成的可执行文件体积很小（部分链接，保留动态重定位信息）；运行时由动态链接器装入 `libvector.so` 并解析。

- 运行时按需加载

```
void *h = dlopen("./libvector.so", RTLD_LAZY);
void (*addvec)(int*,int*,int*,int) = dlsym(h, "addvec");
dlclose(h);
```

五 不同时段（编译时、加载时、运行时）的动态链接的实现

方式

编译时：构建可被动态链接的库

目标：把源代码做成位置无关、可供别人在以后（加载/运行时）装入的 `.so`。

最小流程

```
# 1) 编译为位置无关代码 (PIC)
gcc -fPIC -c addvec.c multvec.c

# 2) 生成共享库 (推荐写 SONAME)
gcc -shared -Wl,-soname,libvector.so.1 -o libvector.so.1.0 addvec.o multvec.o
ln -s libvector.so.1.0 libvector.so.1
ln -s libvector.so.1 libvector.so
```

要点

- **-fPIC/-fpic**：生成位置无关代码，便于库被映射到任意地址并被多个进程共享代码页。
- **-shared**：产出 ELF 类型为共享对象；写入动态段、导出符号等。
- **SONAME**：固定 ABI 名（如 `libx.so.1`），便于升级到 `libx.so.1.2` 而不需重链应用。

应用在链接阶段只记录“我依赖哪个库/SONAME”，并不把库代码放进可执行文件：

```
gcc -o prog main.o -L. -lvector      # 记录 DT_NEEDED: libvector.so.1
```

生成的可执行文件包含：

- `.interp`：指定动态链接器（如 `/lib64/ld-linux-x86-64.so.2`）
- `.dynamic`：依赖库（`DT_NEEDED`）、`RPATH/RUNPATH`、重定位表等

加载时（Load-time）动态链接：进程启动时由动态链接器完成

当执行 `./prog`：

- 1. **定位库**：动态链接器按 `RPATH/RUNPATH` → `LD_LIBRARY_PATH` → 系统目录 顺序找 `DT_NEEDED` 的 `.so`。
- 2. **映射**：把可执行文件与各 `.so` 的代码/数据段 `mmap` 到进程；代码段通常**可共享、只读**。
- 3. **重定位**：根据 `.rel[a].dyn` 等修补 GOT/数据引用、函数指针等地址。
- 4. **PLT/GOT（延迟绑定）**：外部函数第一次调用经 PLT 触发解析，把真实地址写回 GOT；之后直接跳转（可用 `LD_BIND_NOW=1` 改为立即绑定）。
- 5. **运行构造器**：处理 TLS，按依赖顺序执行库的 `.init_array`，最后跳到 `main`。

效果：**可执行体很小**（只含依赖信息），多个进程**共享同一份库代码页**，库升级（ABI 不变）应用通常无需重链。

三、运行时（Run-time）动态链接：程序主动按需加载

程序在运行过程中显式装载/查找/卸载共享库，常用于插件、A/B、按需功能等。

代码与构建

```
// d11.c
void *h = dlopen("./libvector.so.1", RTLD_LAZY);    // 定位并映射库
void (*addvec)(int*,int*,int*,int) = dlsym(h, "addvec"); // 查符号
addvec(x,y,z,2);
dlclose(h);
gcc -o prog2r d11.c -ldl # 使用 dlopen/dlsym 需链接 libdl
# 若库要回调到程序里的全局符号，需：
gcc -rdynamic -o prog2r d11.c -ldl # 导出可执行中的符号给动态链接器可见
```

运行路径：应用通过 `dlopen` → 动态链接器定位/映射库 → `dlsym` 查符号 → 首次调用仍可走 PLT/GOT 的**延迟解析**。

对比

维度	编译时（构建库）	加载时动态链接（进程启动）	运行时动态链接（程序主动）
触发者	开发者/构建系统	内核 <code>execve</code> → 动态链接器	应用代码调用 <code>dlopen/dlsym</code>
发生时 机	构建阶段	启动到 <code>main</code> 之前	程序运行中任意时刻
产物/状 态	<code>.so</code> （PIC、导出符 号、SONAME）	映射所有 <code>DT_NEEDED</code> 库，完成必要 重定位，设置 PLT/GOT	按需映射指定库，按需解析被请 求的符号
灵活性	——	中等（依赖在链接时固定）	最高（可按需加载/卸载、切换实 现、插件架构）
启动/运 行性能	——	启动可能有解析成本；延迟绑定可 把成本推迟到首次调用	启动快；首次 <code>dlopen/dlsym</code> 有额外成本
资源占 用	——	代码段页可在多个进程间共享（省 内存/缓存）	同左；还能只在需要时加载库 （更省内存）

维度	编译时（构建库）	加载时动态链接（进程启动）	运行时动态链接（程序主动）
失败模式	——	找不到库/版本不符 → 程序启动失败	<code>dlopen</code> 失败只影响使用该功能的路径，可降级处理
常见选项	<code>-fPIC -shared -w1, -soname</code>	<code>-w1, -rpath/RUNPATH, LD_LIBRARY_PATH, LD_BIND_NOW</code>	<code>-ld1, -rdynamic</code> （若需让库解析到可执行体里的符号）

六 静态链接和动态链接各自的优缺点

- 静态的优势
 - 运行时**不查库、不走 PLT 间接**；路径最短、依赖最少。
 - 交付一个可执行文件即可跑。
- 静态的劣势
 - 磁盘/内存浪费**：每个程序各带一份相同库代码。
 - 维护成本高**：系统库小修复也要**重链所有依赖程序**。
- 动态的优势
 - 更省空间/内存**：共享库的代码页在多个进程之间共享，且经常处于**热缓存**。
 - 易于更新**：替换 `.so` 即可（前提：**ABI 兼容**）。
 - 灵活**：支持 `dlopen` 运行时加载、`LD_PRELOAD` 拦截、A/B 实现切换。
- 动态的劣势
 - 查找与间接开销**：启动需要解析依赖；函数首次调用可能触发**延迟绑定**；后续通过 GOT 间接跳转。
 - 运维复杂度**：要处理库版本和搜索路径；版本不匹配会导致启动失败。

维度	静态链接 (.a)	动态链接 (.so)
运行依赖	生成的可执行文件 自包含 ，运行时不再依赖库文件	只记录 依赖关系 ，启动/运行时需能找到对应 <code>.so</code>
体积 & 内存	可执行体 更大 ；每个进程各自带一份库代码， 内存总占用高	可执行体 小很多 （示例：hello world 7K vs 571K）；库代码段可在进程间 共享映射 ，内存友好
性能路径	无库查找；无 PLT/GOT 额外间接跳转 → 调用开销略低，启动解析成本低	需要 库定位/重定位 ；外部函数经 PLT/GOT ，有 查找与间接跳转 开销；可用 延迟绑定 把成本推迟到首次调用
更新与维护	库修补（含安全补丁）→ 每个应用都要重链/重发	库升级 即可惠及所有应用 （ABI 兼容前提下）， 打补丁更快
可移植/部署	单文件部署 、离线环境友好；容器/嵌入式常用	需处理 库搜索路径/版本 （ <code>RPATH/RUNPATH</code> 、 <code>LD_LIBRARY_PATH</code> 、 <code>ldconfig</code> 等）
符号拦截/插件	很难在运行时替换实现	支持 <code>dlopen/dlsym</code> 插件化；可 <code>LD_PRELOAD</code> / 运行时 库插桩/拦截

维度	静态链接 (.a)	动态链接 (.so)
可复现性	二进制更可复现（库被烘焙进来）	受系统库版本影响，跨机行为差异可能更大
链接器细节	命令行顺序敏感（对象在前、库在后；否则“undefined reference”）	构建库需 <code>-fPIC -shared</code> ；应用常规链接记录 <code>DT_NEEDED</code> ，运行时由动态链接器处理
典型场景	静态工具链、启动盘、空气隔离环境、对启动/调用开销极敏感且功能固定的程序	桌面/服务器软件、需要更小发布体积、热修复/快速更新、插件化/按需加载的系统

七 编译时“库插桩”的具体操作

编译时库插桩 = “同名头 + 宏重定向 + 包装实现”。通过 `-I.` 先让编译器找到我们的 `malloc.h`，把 `malloc/free` 在编译期替换为 `mymalloc/myfree`，运行时即可无侵入地观测分配与释放。

目标

- 不改动程序语义的前提下，记录每次 `malloc/free` 的尺寸与地址。
- 方案：在编译阶段把源代码里的 `malloc/free` 宏替换成我们自定义的包装函数（wrapper），包装里完成打印/统计，再转调“真”`malloc/free`。

示例驱动（课件里的 `int.c` 思路）：

```
#include <stdio.h>
#include <malloc.h>    // 将被我们“同名头文件”覆盖
#include <stdlib.h>

int main(int argc, char *argv[]) {
    for (int i = 1; i < argc; i++) {
        void *p = malloc(atoi(argv[i]));
        free(p);
    }
    return 0;
}
```

机制（Compile-time Interpositioning）

核心是头文件劫持 + 宏替换：

- 提供一个同名头文件 `malloc.h`，里面把

```
#define malloc(size) mymalloc(size)
#define free(ptr)    myfree(ptr)
```

- 再实现 `mymalloc/myfree` 两个函数；它们内部最终仍调用真正的 `malloc/free` 并打印信息。

- 编译器选项 `-I.` 把当前目录加入尖括号包含的查找路径，使 `#include <malloc.h>` 先命中我们的头，从而在编译期完成替换。

文件与代码

```
/* malloc.h -名字与系统头相同 */
#ifndef MY_MALLOC_H
#define MY_MALLOC_H
#include <stddef.h>
void *mymalloc(size_t size);
void myfree(void *ptr);
#define malloc(size) mymalloc(size)
#define free(ptr) myfree(ptr)
#endif
```

mymalloc.c

```
#ifdef COMPILETIME
#include <stdio.h>
#include <stdlib.h> // 真正的 malloc/free 声明

/* malloc 包装函数 */
void *mymalloc(size_t size) {
    void *ptr = malloc(size); // 调用“真”malloc
    printf("malloc(%d)=%p\n", (int)size, ptr);
    return ptr;
}

/* free 包装函数 */
void myfree(void *ptr) {
    free(ptr); // 调用“真”free
    printf("free(%p)\n", ptr);
}
#endif
```

编译与运行

```
# 1) 先把包装实现编译成目标文件，并打开编译期开关
gcc -Wall -DCOMPILETIME -c mymalloc.c

# 2) 再编译链接示例程序；用 -I. 让 <malloc.h> 命中同名头
gcc -Wall -I. -o intc int.c mymalloc.o

# 3) 运行验证（参数是每次分配的字节数）
./intc 10 100 1000
```

输出：

```
malloc(10)=0x1ba7010
free(0x1ba7010)
malloc(100)=0x1ba7030
free(0x1ba7030)
malloc(1000)=0x1ba70a0
free(0x1ba70a0)
```

八 链接时“库插桩”的具体操作

链接时插桩利用 ld 的 `--wrap` 机制，在**不改业务源码**的情况下，把对目标符号（如 `malloc/free`）的调用**统一改道**到 `__wrap_*`，再通过 `__real_*` 调回真实实现，从而在**链接阶段**完成“掉包”拦截与监控。

写两个**包装实现**：`__wrap_malloc` / `__wrap_free`。

在包装里，真正想调用 libc 的实现时，用“对偶符号”`__real_malloc` / `__real_free`。

链接器选项 `-Wl,--wrap,symbol` 规定：

- 所有对 `symbol` 的引用（如 `malloc`）都**改指向** `__wrap_symbol`（`__wrap_malloc`）。
- 对 `__real_symbol` 的引用（`__real_malloc`）则**改回** `symbol`（真正的 `malloc`）。

九 加载/运行时“库插桩”的具体操作

加载/运行时插桩通过 `LD_PRELOAD` + “同名函数”实现运行时劫持。

先把拦截实现做成**共享库**（`.so`），里面定义与**被拦函数同名**的实现（如 `malloc/free`）。

动态链接器在**装载可执行文件**时会先加载 `LD_PRELOAD` 指定的 `.so`，当解析到 `malloc/free` 时，优先使用 `.so` 里的同名符号**。

内部再用 `dlsym(RTLD_NEXT, "malloc")` / `dlsym(RTLD_NEXT, "free")` 找到“后继”的真实实现（通常是 libc），完成转调。